

```

# The exercise 1
# Luyi Huang 9/3/2026

# In this exercise, I use data that recorded the MORT values vary with different
# POOR values, that is, the Total age-adjusted mortality rate per 1000 000 varies
# when the area has corresponding % of families with income<3000$. I try to use local
# linear regression method to do the prediction when POOR values are 10, 18,25 and
# the respective expected value variances are also calculated.
import pandas as pd
import numpy as np
import matplotlib
from numpy import array
from matplotlib import pyplot as plt

# Extract data from csv document and sort by "POOR" values
# Extract "POOR" and "MORT" variables
data=pd.read_csv("pollution_cleaneddata.csv")
data_sort=data.sort_values(by="POOR")
MORT=data_sort['MORT']
POOR=data_sort['POOR']
MORT_array=MORT.to_numpy()
POOR_array=POOR.to_numpy()

print(POOR_array)
plt.plot(POOR_array,MORT_array,'.')
plt.xlabel('% of families with income<3000$')
plt.ylabel('Total age-adjusted mortality rate per 1000 000')
plt.legend()
plt.show()

# The weighted function of k is modelled by Gaussian distribution function
#  $W(d)=\exp^{(-d^2/(2*\sigma^2))}$ , where sigma was calculated by the distance D between x0 and
# the furthest point from the x0 within these k points, that is,
#  $3*\sigma=\text{distance } D(\text{x0 with furthest point within k points})$ 
def pred_single(y,x,k,x0):
    i=0
    while (x[i]-x0)*(x[i+1]-x0)>0:
        i=i+1
    if (i-k//2+1)>=0 and (i+k//2)<60:
        D=max(x0-x[i-k//2+1],x[i+k//2]-x0) #find which side is farer from x0
        sigma=D/3
        x_sum=0
        y_sum=0
        j=0
        while j<k:
            x_sum+=x[i-k//2+1+j]*np.exp(-(x0-x[i-k//2+1+j])**2/(2*sigma**2))
            y_sum+=y[i-k//2+1+j]*np.exp(-(x0-x[i-k//2+1+j])**2/(2*sigma**2))
            j=j+1
        x_average=x_sum/k
        y_average=y_sum/k
        j=0
        b1_denominator=0
        b1_numerator=0
        while j<k:
            b1_numerator+=(x[i-k//2+1+j]*np.exp(-(x0-x[i-k//2+1+j])**2/(2*sigma**2))-x_average)*(y[i-
k//2+1+j]*np.exp(-(x0-x[i-k//2+1+j])**2/(2*sigma**2))-y_average)
            b1_denominator+=(x[i-k//2+1+j]*np.exp(-(x0-x[i-k//2+1+j])**2/(2*sigma**2))-x_average)**2
            j+=1
        b1_estimate=b1_numerator/b1_denominator
        b0_estimate=y_average-b1_estimate*x_average
    elif (i-k//2+1)<0:
        D=x[k-1]-x0
        sigma=D/3

```

```

x_sum=0
y_sum=0
j=0
while j<k:
    x_sum+=x[j]*np.exp(-(x0-x[j])**2/(2*sigma**2))
    y_sum+=y[j]*np.exp(-(x0-x[j])**2/(2*sigma**2))
    j=j+1
x_average=x_sum/k
y_average=y_sum/k
j=0
b1_denominator=0
b1_numerator=0
while j<k:
    b1_numerator+=(x[j]*np.exp(-(x0-x[j])**2/(2*sigma**2))-x_average)*(y[j]*np.exp(-(x0-
x[j])**2/(2*sigma**2))-y_average)
    b1_denominator+=(x[j]*np.exp(-(x0-x[j])**2/(2*sigma**2))-x_average)**2
    j+=1
b1_estimate=b1_numerator/b1_denominator
b0_estimate=y_average-b1_estimate*x_average
else:
    D=x0-x[59-k]
    sigma=D/3
    x_sum=0
    y_sum=0
    j=59-k
    while j<60:
        x_sum+=x[j]*np.exp(-(x0-x[j])**2/(2*sigma**2))
        y_sum+=y[j]*np.exp(-(x0-x[j])**2/(2*sigma**2))
        j=j+1
    x_average=x_sum/k
    y_average=y_sum/k
    j=59-k
    b1_denominator=0
    b1_numerator=0
    while j<60:
        b1_numerator+=(x[j]*np.exp(-(x0-x[j])**2/(2*sigma**2))-x_average)*(y[j]*np.exp(-(x0-
x[j])**2/(2*sigma**2))-y_average)
        b1_denominator+=(x[j]*np.exp(-(x0-x[j])**2/(2*sigma**2))-x_average)**2
        j+=1
    b1_estimate=b1_numerator/b1_denominator
    b0_estimate=y_average-b1_estimate*x_average

    return b0_estimate+b1_estimate*x0
def pred(y,x,k,x0_vec):
    return pred_single(y,x,k,x0_vec[0]),pred_single(y,x,k,x0_vec[1]),pred_single(y,x,k,x0_vec[2])

print("the respective values of MORT when POOR values are 10, 18, 25 are")
print(*pred(MORT_array,POOR_array,26,array([10,18,25])))

# When calculating the variance, the weighted function of k points are not considered.
def se_single(y,x,k,x0):
    i=0
    while (x[i]-x0)*(x[i+1]-x0)>0:
        i=i+1
    if (i-k//2+1)>=0 and (i+k//2)<60:
        x_sum=0
        y_sum=0
        j=0
        while j<k:
            x_sum+=x[i-k//2+1+j]
            y_sum+=y[i-k//2+1+j]
            j=j+1
        x_average=x_sum/k
        y_average=y_sum/k
        j=0

```

```

b1_denominator=0
b1_numerator=0
while j<k:
    b1_numerator+=(x[i-k//2+1+j]-x_average)*(y[i-k//2+1+j]-y_average)
    b1_denominator+=(x[i-k//2+1+j]-x_average)**2
    j+=1
b1_estimate=b1_numerator/b1_denominator
b0_estimate=y_average-b1_estimate*x_average
error_square=0
x_square=0
j=0
while j<k:
    error_square+=(y[i-k//2+1+j]-b1_estimate*x[i-k//2+1+j]-b0_estimate)**2
    x_square+=(x[i-k//2+1+j]-x_average)**2
    j+=1
expected_value_variance=error_square/(k-2)*(1/k+(x0-x_average)**2/x_square)
elif (i-k//2+1)<0:
    x_sum=0
    y_sum=0
    j=0
    while j<k:
        x_sum+=x[j]
        y_sum+=y[j]
        j=j+1
    x_average=x_sum/k
    y_average=y_sum/k
    j=0
    b1_denominator=0
    b1_numerator=0
    while j<k:
        b1_numerator+=(x[j]-x_average)*(y[j]-y_average)
        b1_denominator+=(x[j]-x_average)**2
        j+=1
    b1_estimate=b1_numerator/b1_denominator
    b0_estimate=y_average-b1_estimate*x_average
    error_square=0
    x_square=0
    j=0
    while j<k:
        error_square+=(y[j]-b1_estimate*x[j]-b0_estimate)**2
        x_square+=(x[j]-x_average)**2
        j+=1
    expected_value_variance=error_square/(k-2)*(1/k+(x0-x_average)**2/x_square)
else:
    x_sum=0
    y_sum=0
    j=59-k
    while j<60:
        x_sum+=x[j]
        y_sum+=y[j]
        j=j+1
    x_average=x_sum/k
    y_average=y_sum/k
    j=59-k
    b1_denominator=0
    b1_numerator=0
    while j<60:
        b1_numerator+=(x[j]-x_average)*(y[j]-y_average)
        b1_denominator+=(x[j]-x_average)**2
        j+=1
    b1_estimate=b1_numerator/b1_denominator
    b0_estimate=y_average-b1_estimate*x_average
    error_square=0
    x_square=0
    j=59-k

```

```
    while j<60:
        error_square+=(y[j]-b1_estimate*x[j]-b0_estimate)**2
        x_square+=(x[j]-x_average)**2
        j+=1
    expected_value_variance=error_square/(k-2)*(1/k+(x0-x_average)**2/x_square)

    return expected_value_variance
def se(y,x,k,x0_vec):
    return se_single(y,x,k,x0_vec[0]),se_single(y,x,k,x0_vec[1]),se_single(y,x,k,x0_vec[2])

print("the respective values of expected value variance when POOR values are 10, 18, 25 are")
print(*se(MORT_array,POOR_array,26,array([10,18,25])))

# Conclusion:
# The result reflect a pattern that the poorer an area is, the higher the
# Total age-adjusted mortality rate per 1000 000 is.
```